HYUNDAI AI Insight Campus · 온디바이스 AI

Day 4 버스 통신 — I2C · SPI · UART

Embedded I/O 제어 4/5 · 10/16(금) · Raspberry Pi 4 + /dev/i2c-1 · /dev/spidev0.0 · /dev/serial0

강의 자료 · 애니메이션 · 워크시트

오늘 만드는 것

Day 3 → Day 4

핀 하나에 0/1 · PWM 을 내던 것에서
선 두세 개로 바이트를 주고받는 " 버스 " 로 .

미니 프로젝트 — gateway.c

ESP32 DHT11 값을 UART 로 받아 7-Segment 에 표시
조도가 어두우면 LED 명령을 되돌려 보냄

1 09:15 **I2C 와 PCF8591**

2 10:00 **DAC 와 다채널 읽기**

3 11:00 **아날로그 센서 확장**

4 13:00 **SPI 와 74HC595**

5 14:00 **4 자리 7-Segment 멀티플렉싱**

6 15:00 **8×8 도트 매트릭스**

7 16:00 **UART 와 termios**

8 17:00 **미니 프로젝트 — 센서 게이트웨이**

세 버스 — 모두 파일이다

버스	장치 파일	설정	주고받기	커널 드라이버
I2C	/dev/i2c-1	ioctl(I2C_SLAVE, 0x48)	write · read · ioctl(I2C_RDWR)	i2c-bcm2835
SPI	/dev/spidev0.0	ioctl(SPI_IOC_WR_MODE ...)	ioctl(SPI_IOC_MESSAGE(1))	spi-bcm2835
UART	/dev/serial0	tcsetattr (termios)	read · write	amba-pl011

BCM (물리)	오늘
GPIO2 · 3 (3 · 5)	I2C SDA · SCL — PCF8591
GPIO10 · 11 · 8 (19 · 23 · 24)	SPI MOSI · SCLK · CE0 — 74HC595
GPIO14 · 15 (8 · 10)	UART TXD · RXD — ESP32
GPIO17 · 27 · 22 · 5	4 자리 자리 선택 / 매트릭스 행은 5 · 6 · 12 · 13 · 16 · 20 · 21 · 26

I2C — 두 선, 주소, ACK

1교시

용어	뜻
SDA · SCL	데이터 · 클럭 두 선에 여러 장치
주소	7 비트 — PCF8591 = 0x48
풀업	평소 High, 장치는 Low 로만 당김
ACK	받을 때마다 SDA 를 Low 로 "받았다"

```
$ i2cdetect -y 1
40: -- -- -- -- -- -- -- -- 48 --

$ i2cget -y 1 0x48 0x40   # 이전 값
$ i2cget -y 1 0x48 0x40   # 지금 값
$ i2cset -y 1 0x48 0x40 200  # DAC
```

⚠ 모듈 Vcc 는 반드시 3V3

기판 풀업이 Vcc 로 버스를 당긴다 → 5V 면 SDA · SCL 이 5V.
윗줄 핀 순서 : Scl · Sda · Vcc · Gnd

첫 바이트는 이전 변환 값

읽기 요청을 받는 순간 다음 변환을 시작 → 2 바이트 읽고 buf[1] 사용.

애니메이션 D4-1 • I2C 트랜잭션 ▶

C 로 — open · ioctl · write · read

1교시

```
int fd = open("/dev/i2c-1", O_RDWR);
ioctl(fd, I2C_SLAVE, 0x48);

uint8_t ctrl = 0x40 | ch;      /* DAC 켜기 + 채널 */
write(fd, &ctrl, 1);
uint8_t buf[2];
read(fd, buf, 2);
return buf[1];                 /* 지금 값 */
```

MCU (Wire)	Linux
Wire.begin()	open("/dev/i2c-1")
beginTransmission(0x48)	ioctl(I2C_SLAVE)
write + endTransmission	write(fd, ...)
requestFrom + read	read(fd, ...)

반환값을 검사하지 않으면 선이 빠져도 이전 값 · 255 를 그대로 믿게 된다 → ENXIO 확인

제어 바이트 · DAC · 자동 증가

2 교시

비트	7	6	5 · 4	3	2	1 · 0
뜻	0	DAC 켜기	입력 방식	0	자동 증가	채널

하고 싶은 일	바이트
AIN0 읽기	0x40
AIN2 읽기	0x42
네 채널 자동 증가	0x44 → 5 바이트 읽기
DAC = v	0x40, v

```
struct i2c_msg m[2] = {
  { 0x48, 0,        1, &ctrl },  /* 0x44 */
  { 0x48, I2C_M_RD, 5, buf   } };
ioctl(fd, I2C_RDWR, &data);
/* START … repeated START … STOP */
```

03_joy_servo — MCU Day 2 를 Linux 로

조이스틱 VRx · VRy → IN1 · IN2 (3V3 전원) · SW → GPIO6 · 서보 → GPIO18 (Day 3 pwm_sysfs.h)

애니메이션 D4-2 • 제어 바이트와 자동 증가 ▶

꽂기 전에 계산 — 3.3 V 를 넘는가

3 교시

센서	연결	최대	판정
포토레지스터	3V3 – R – IN0 – 10kΩ – GND	< 3.3 V	✅
LM35	+Vs 5V · Vout → IN1	1.5 V	✅
불꽃 (2 핀)	– → 3V3 · + → IN2 · 10kΩ → GND	< 3.3 V	✅
수위	+ → GPIO25 · S → IN3	3.3 V	✅
MQ-2	AO	5 V	❌ 오늘 안 씀

12.9 mV

8 비트 한 칸 (3300 / 255)
LM35 기준 약 1.3 °C

04_sensors

이동 평균 8 개 (링 버퍼)
불꽃 경보 히스테리시스 120 / 90
수위 센서는 잴 때만 전원

애니메이션 D4-3 • 분압과 물리량 변환 ▶

SPI → 74HC595 — CE0 가 래치다

4 교시

SPI	Pi	595	역할
MOSI	GPIO10 (19)	DS 14	비트
SCLK	GPIO11 (23)	SH_CP 11	상승마다 한 칸 밀기
CE0	GPIO8 (24)	ST_CP 12	전송 끝 상승 = 래치
—	3V3 · GND	MR 10 · OE 13	MR → 3V3, OE → GND

```
fd = open("/dev/spidev0.0");
mode = SPI_MODE_0;
ioctl(fd, SPI_IOC_WR_MODE, &mode);

tr.tx_buf = &b; tr.len = 1;
ioctl(fd, SPI_IOC_MESSAGE(1), &tr);
```

MSB 먼저

bit7 → Q7(dp) … bit0 → Q0(a)
"0" = 0x3F (MCU Day 3 과 같은 글꼴)

세그먼트마다 1kΩ

4 · 5 · 6 · 8 교시 같은 배선 — 이유는 5 교시 전류 계산

애니메이션 D4-4 • SPI 시프트 • 래치 ▶

4 자리 멀티플렉싱 — 자리 핀 전류부터

5 교시

	220 Ω	1 kΩ
세그먼트 하나	5.9 mA	1.3 mA
"8." → 자리 핀 하나	47 mA ×	10 mA ✓
GPIO 권장	16 mA	16 mA

```
/* 자리 k — 2 ms 마다 */
all_digits_off();          /* ① */
spi_send(spi, buf[k]);     /* ② */
digit_on(k);               /* ③ active-low */
clock_nanosleep(…, TIMER_ABSTIME, …);
```

상황	최대 지연	보임
부하 없음	약 80 µs	깨끗
yes × 4	약 9 ms	한 자리 번쩍
yes × 4 + SCHED_FIFO	약 60 µs	깨끗

애니메이션 D4-5 • 멀티플렉싱과 잔상 ▶

8×8 매트릭스 — 행 스캔 · rev8 · 이중 버퍼

6 교시

1088AS 핀	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
신호	R5	R7	C2	C3	R8	C5	R6	R3	R1	C4	C6	R4	C1	R2	C7	C8

행	R1	R2	R3	R4	R5	R6	R7	R8
GPIO	5	6	12	13	16	20	21	26

rev8()

그림 bit7 = 왼쪽 (C1 = Q0)
SPI 는 MSB 먼저 → 뒤집어 보냄

버퍼 1장

스캔 도중 그림이 바뀌면
위 행 = 옛 그림 · 아래 행 = 새 그림 (찢어짐)

이중 버퍼

메인은 뒤 버퍼에 그리고 교체 요청
스캔 쓰레드는 R1 직전에만 교체

애니메이션 D4-6 • 매트릭스 편집기와 이중 버퍼 ▶

UART — 클럭 없이 약속한 속도로

7 교시

```
쉼 ‾‾\_S_b0_b1_..._b7_P‾‾  쉼
115200 8N1: 비트 8.7 µs · 글자 10 비트
→ 초당 약 11 520 바이트
```

UART	클럭	오늘
PL011 ttyAMA0	정확	disable-bt → GPIO14 · 15
mini ttyS0	CPU 따라 변함	쓰지 않음

termios	뜻
cfmakeraw	에코 · 줄 편집 · 특수 문자 끔
CS8 · ~PARENB · ~CSTOPB	8N1
CLOCAL | CREAD · ~CRTSCTS	모뎀 선 무시 · 흐름 제어 끔
VMIN 0 · VTIME 1	있으면 바로 , 없으면 0.1 초 뒤 0

ESP32 연결

Pi TXD(8) → RX2(16)
Pi RXD(10) ← TX2(17)
GND ↔ GND (필수)
3V3 · 5V 는 잇지 않음

애니메이션 D4-7 • UART 프레임과 VMIN • VTIME ▶

루프백 먼저 — 08_uart_term -t

7 교시

```
$ ./08_uart_term -t      # TXD ↔ RXD 선 하나
← ping 1      0.71 ms
← ping 2      0.69 ms
   (응답 없음 — TX·RX 연결 확인)
보냄 3 · 받음 2 · 응답 없음 1
```

한 줄 ≠ read() 한 번

"T=23" 과 ".0,H=45\n" 처럼 쪼개져 온다
→ lb_feed() 줄 버퍼에 모아 \n 에서 처리

"ping 1\n" 7 바이트 × 10 비트 / 115200 ≈ 0.61 ms — 측정값과 거의 같다

게이트웨이 프로토콜 — 한 줄 + XOR 체크섬

8 교시

방향	줄	뜻
ESP32 → Pi	T=23.0,H=45*2E	1 초마다 DHT11
ESP32 → Pi	ERR=DHT*..	읽기 실패
Pi → ESP32	LED=1*41 / LED=0*40	어두우면 LED

```
"LED=1" → 0x4C ^ 0x45 ^ 0x44 ^ 0x3D ^ 0x31 = 0x41 → "LED=1*41"
```

받는 쪽 규칙

체크섬 틀림 → 버림 · 형식 · 범위 틀림 → 버림 · 3 초 동안 정상 없음 → "----" · 조도 30 % 아래 LED=1 / 40 % 위 LED=0

애니메이션 D4-8 • 게이트웨이 시뮬레이터 ▶

gateway.c 구조 — epoll + 표시 쓰레드

8 교시

누가	하는 일
epoll — UART fd	바이트 → 줄 버퍼 → 체크섬 · 형식 → 값 저장
epoll — timerfd 1 초	조도 → LED 명령 · 화면 내용 · 3 초 타임아웃
seg4 쓰레드	4 자리 표시 (5 교시 그대로)

실행	용도
./gateway	ESP32 연결
./gateway -s	루프백 선 + 가짜 프레임
./gateway -k	키보드로 프레임
./gateway -f	표시 SCHED_FIFO

확인할 것

"23.0" ↔ "H 45" 번갈아 · 포토레지스터 가리면 ESP32 LED 켜짐 · -k 로 T=23.0,H=45*00 → 체크섬 버림 · ESP32 끄면 3 초 뒤 "----"

오늘 정리

세 버스 모두 파일 · I2C = 주소 + ACK, 첫 바이트는 이전 값 · 입력 최대 전압부터 계산 · SPI 전송 끝 CE0 = 래치 · 멀티플렉싱은 끄기 → 모양 → 켜기 + 자리 핀 전류 · 이중 버퍼 · UART = 같은 baud + termios raw + 줄 버퍼 + 체크섬

다음 시간 (Day 5, 10/19 월) DHT11 커널 드라이버 (IIO) · Device Tree overlay 직접 작성 · 1602A LCD 드라이버 · 종합 프로젝트 envmon + systemd

워크시트 풀기 ▶